Devoir n°1 : Patrons de construction (INF461)

AMAYEN BOUSSIOM THIERRY MARTIN
Matricule : 25I2177

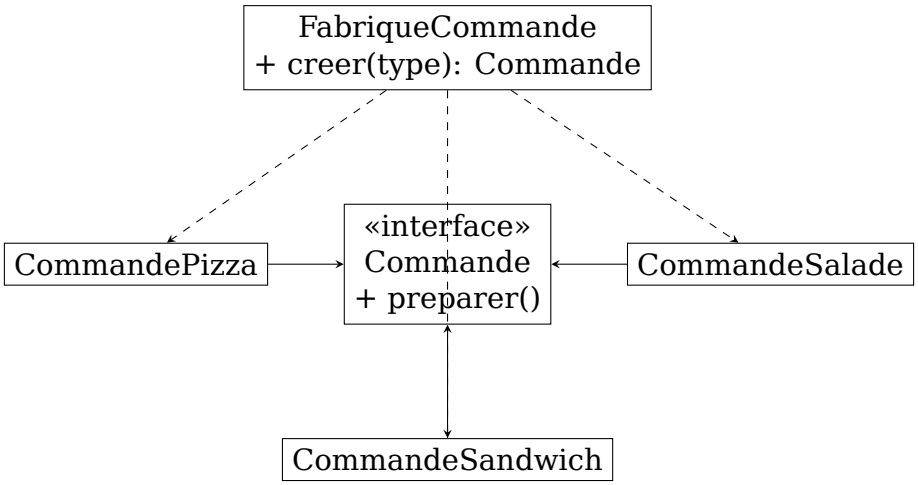
Introduction

Ce devoir présente deux exemples personnels des patrons Factory Method et Factory. Chaque exemple comporte d'abord trois classes concrètes, puis une version modifiée avec une quatrième classe. Les diagrammes montrent les relations entre le client, les fabriques et les objets créés.

1 Factory Method : commandes de restaurant

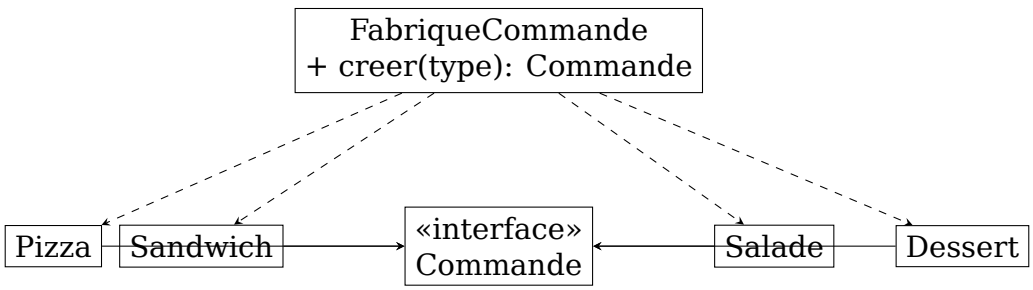
Une application de restauration crée une commande à partir du choix du client. La fabrique unique connaît les types de commandes.

1.1 Exemple initial : trois commandes



1.2 Variante : quatre commandes

La classe `CommandeDessert` est ajoutée. `FabriqueCommande` reçoit aussi le type `dessert` ; la fabrique doit donc être modifiée.

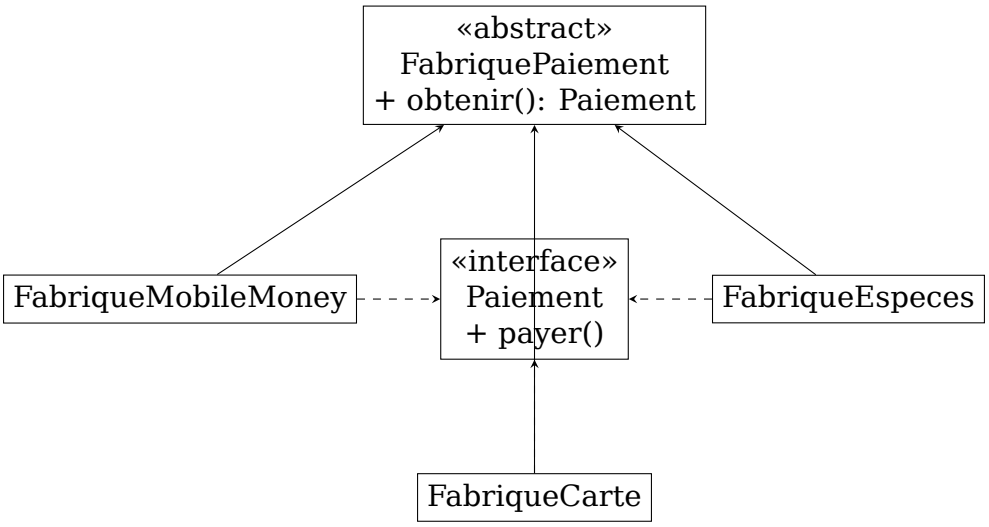


```
interface Commande { String preparer(); }
class CommandePizza implements Commande { public String preparer(){ return "Pizza"; } }
class CommandeSandwich implements Commande { public String preparer(){ return "Sandwich"; } }
class CommandeSalade implements Commande { public String preparer(){ return "Salade"; } }
class CommandeDessert implements Commande { public String preparer(){ return "Dessert"; } }
class FabriqueCommande { Commande creer(String t) { return switch(t) {
 case "pizza" -> new CommandePizza(); case "sandwich" -> new CommandeSandwich();
 case "salade" -> new CommandeSalade(); case "dessert" -> new CommandeDessert(); default -> throw new
      IllegalArgumentException(); }; } }
```

2 Factory : moyens de paiement

Une application de vente utilise une fabrique abstraite. Chaque sous-classe crée son propre moyen de paiement.

2.1 Exemple initial : trois paiements



2.2 Variante : quatre paiements

On ajoute `VirementBancaire` et `FabriqueVirement`. Les fabriques existantes restent inchangées : c'est l'avantage du patron Factory.

```
interface Paiement { String payer(); }
abstract class FabriquePaiement { Paiement obtenir(){ return creer(); } protected abstract Paiement creer(); }
class MobileMoney implements Paiement { public String payer(){ return "Mobile␣Money"; } }
class CarteBancaire implements Paiement { public String payer(){ return "Carte␣bancaire"; } }
class Especes implements Paiement { public String payer(){ return "Espèces"; } }
class VirementBancaire implements Paiement { public String payer(){ return "Virement␣bancaire"; } }
class FabriqueMobileMoney extends FabriquePaiement { protected Paiement creer(){ return new MobileMoney(); } }
class FabriqueCarte extends FabriquePaiement { protected Paiement creer(){ return new CarteBancaire(); } }
class FabriqueEspeces extends FabriquePaiement { protected Paiement creer(){ return new Especes(); } }
class FabriqueVirement extends FabriquePaiement { protected Paiement creer(){ return new VirementBancaire(); } }
```

3 Analyse et dépôt GitHub

Factory Method centralise la création dans une seule fabrique ; ajouter un nouveau type impose de modifier cette fabrique. Factory permet d'ajouter une nouvelle fabrique concrète sans modifier les classes existantes. Le dépôt GitHub public doit contenir les codes complets et les diagrammes Dia.